GRUNDLAGEN DER DIGITALISIERUNG UND DIGITALEN TRANSFORMATION

Übung 1 - Vorbereitung SQL



LERNZIELE

- Dieses Assignment hat das Ziel, Sie mit den Konzepten von SQL (Structured Query Language) vertraut zu machen
- Sie finden in diesem Dokument Definitionen und Beispielstatements
- Dieses Dokument enthält außerdem kleine Aufgaben (die Sie nicht bearbeiten müssen) und deren Lösung, um Ihnen anhand einfacher Beispiele SQL näher zu bringen
- Dieses Dokument soll als Unterstützung dienen, um das Assignment zu SQL dieser Woche zu bearbeiten. Sie können allerdings gerne auch andere Informationsquellen nutzen, um SQL zu lernen.
- Nach dem Assignment können Sie selbst einfache Datenbankabfragen in SQL schreiben
- Sie müssen in dieser Woche keine Datenbank aufsetzen oder „echte“ Abfragen schreiben

AGENDA

- Motivation
- Einführung in SQL
- Weitere Informationen

DREI BEISPIELE FÜR WEB-ANWENDUNGEN, DIE AUF DATENBANKEN ZURÜCKGREIFEN

Amazon:

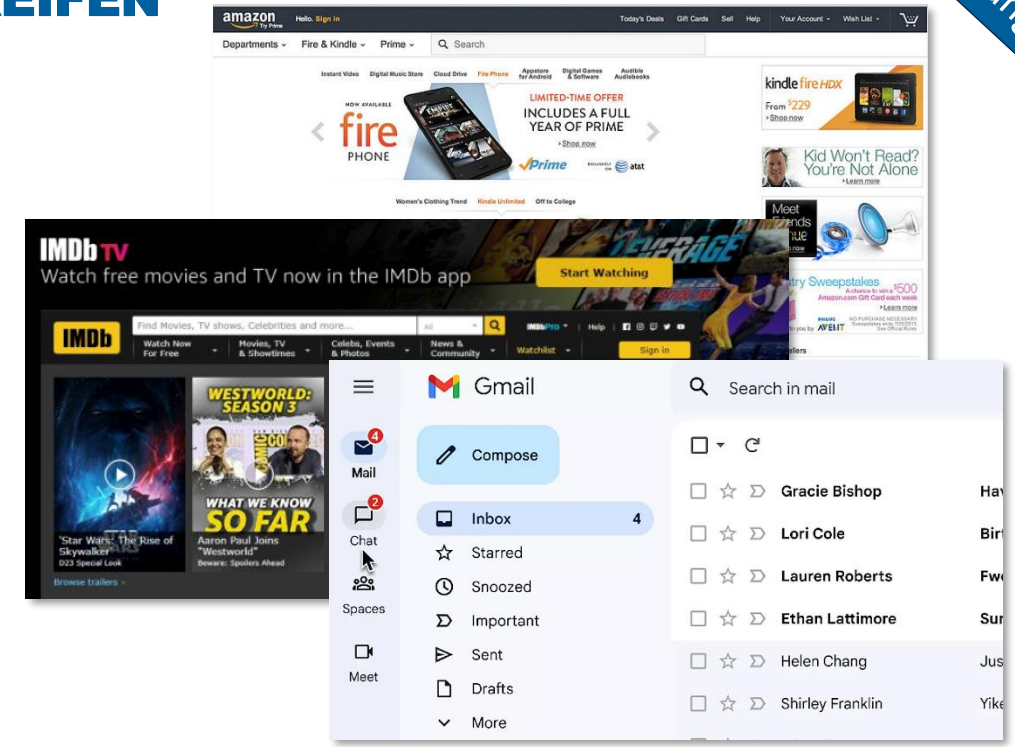
- Speicherung und Verarbeitung von Kundendaten, Bestellinformationen und Transaktionsdaten
- Bereitstellung von Datenbankensystemen für Kunden

Gmail:

- Speicherung und Organisation von Emails
- Weiterentwicklung der Dienstleistung durch Leistungs- und Absturzdaten

IMDb:

- Speicherung von Daten zu Filmen und Serien
- Analyse von Nutzerbewertungen zur Einschätzung der öffentlichen Meinung



DATENBANKEN BILDEN DIE TECHNISCHE GRUNDLAGE FÜR BETRIEBSWIRTSCHAFTLICHE DATENHALTUNG UND -AUSWERTUNG

Relationale Datenbanken werden häufig in Unternehmen eingesetzt und können folgende Daten umfassen:

- Kund*innendaten, z. B. Adressen
- Personaldaten, z. B. Namen von Mitarbeiter*innen, Personalnummer
- Produktdaten, z. B. Zusammensetzung, Preis
- Lieferantendaten, z. B. Namen von Lieferanten

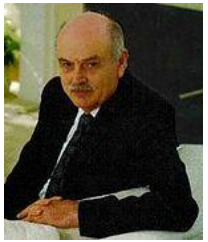
Disziplinen, die solche Daten gewinnbringen nutzen:

- Controlling
- Supply Chain Management
- Marketing
- Vertrieb



GRUNDLAGEN RELATIONALER DATENBANKSYSTEME

- Geht zurück auf die Entwicklung von Edgar Codd aus 1970
- Daten werden in zweidimensionalen Tabellen (=Relationen) dargestellt, die eine feste Anzahl von Spalten und eine beliebige Anzahl von Zeilen haben können.



Begriff	Bedeutung (nicht formal)
Attribut	**Spalte** einer Tabelle
Wertebereich	Mögliche Werte eines Attributs (auch Domäne)
Attributwert	Element des Wertebereichs
Relationsschema	Menge von Attributen
Tupel	**Zeile** einer Tabelle
Relation	Menge von Zeilen einer Tabelle

BEISPIEL ZUM RELATIONALEN MODELL

Relation „Kunde“

Kundennr	Name	Ort
07642	König	Göttingen
63211	Schmitt	Hannover
85861	Müller	Hamburg

AGENDA

- Motivation
- **Einführung in SQL**
- Weitere Informationen

DIE GRUNDIDEE VON SQL

- Structured Query Language (SQL): meistverbreitete deklarative Abfragemethode für relationale Datenbanksysteme
- In einer Datenbank liegen Informationen in Tabellen (ähnlich wie in Excel, aber strukturierter)
- SQL ist eine Sprache, um:
  - Daten anzusehen
  - Daten zu sortieren
  - Daten zu verändern
- Die wichtigsten SQL-Anweisungen im Überblick
  - SELECT → Welche Informationen möchte ich sehen?
  - FROM → Aus welcher Tabelle stammen diese Informationen?
  - WHERE → Welche Datensätze sollen berücksichtigt werden?
  - ORDER BY → Wie sollen die Ergebnisse sortiert werden?

SELECT STATEMENTS

- **Grundform** einer einfachen Abfrage:
  - SELECT Spalte1, Spalte2, …[welche Attribute, z. B. Kundennr., Name]
  - FROM Relation [aus welcher Relationen, z. B. Relation Kunde]
  - WHERE x=...; [unter welchen Bedingungen, z. B. Kundenort = München]

- Beispielabfrage: SELECT Name FROM Kunde WHERE Kundennr=63211;

- **Ausgabe**: Schmitt

Kundennr	Name	Ort
07642	König	Göttingen
63211	Schmitt	Hannover
85861	Müller	Hamburg

WEITERE BEISPIELE ZUR FORMULIERUNG EINFACHER SELECT-STATEMENTS

- Schauen Sie sich die Tabelle und die Statements an. Was denken Sie sind die Ausgaben der jeweiligen Statements? (Die Lösung finden Sie auf der nächsten Folie)

a)
```
SELECT Name
FROM Kunde
WHERE Kundennr = 85861;
```

b)
```
SELECT Ort
FROM Kunde
WHERE Name = 'König';
```

c)
```
SELECT Name
FROM Kunde
WHERE Ort = 'Hannover';
```

d)
```
SELECT Name
FROM Kunde;
```

Kundennr	Name	Ort
07642	König	Göttingen
63211	Schmitt	Hannover
85861	Müller	Hamburg

WEITERE BEISPIELE ZUR FORMULIERUNG EINFACHER SELECT-STATEMENTS

a)
```
SELECT Name
FROM Kunde
WHERE Kundennr = 85861;
```

Ausgabe: Müller

b)
```
SELECT Ort
FROM Kunde
WHERE Name = 'König';
```

Ausgabe: Göttingen

c)
```
SELECT Name
FROM Kunde
WHERE Ort = 'Hannover';
```

Ausgabe: Schmitt

d)
```
SELECT Name
FROM Kunde;
```

Ausgabe:
König
Schmitt
Müller

HINWEIS FÜR SELECT-STATEMENTS

- Wenn Sie selbst SQL-Statements schreiben:
  - Achten Sie bei Attributen, die als Text gespeichert werden, auf die 'ANFÜHRUNGSZEICHEN'
  - Bei Attributen, die als Zahlen gespeichert warden, benötigen Sie diese nicht

ORDER BY

- Die ORDER BY Klausel wird verwendet, um die Ergebnismenge einer SELECT Anweisung entweder in aufsteigender (A-Z, 1-10) oder absteigender (Z-A, 10-1) Reihenfolge zu sortieren.
- Syntax:SELECT Spalte1, Spalte2,… FROM Relation ORDER BY Spalte ASC | DESC;
- Beispielabfrage: SELECT Name, Ort FROM Kunde ORDER BY Name ASC;
- Ausgabe:
  König | Göttingen
  Müller | Hamburg
  Schmitt | Hannover

Kundennr	Name	Ort
07642	König	Göttingen
63211	Schmitt	Hannover
85861	Müller	Hamburg

ORDER BY

- ORDER BY bestimmt, nach welcher Spalte das Ergebnis sortiert wird
- ASC sortiert aufsteigend (A–Z, 1–10)
- DESC sortiert absteigend (Z–A, 10–1)
- Wird keine Sortierreihenfolge angegeben, erfolgt die Sortierung standardmäßig aufsteigend (ASC)

Kundennr	Name	Ort
07642	König	Göttingen
63211	Schmitt	Hannover
85861	Müller	Hamburg

WEITERE BEISPIELE FÜR ODER-BY-STATEMENTS

- Wie würden sie die folgenden SQL-Statements schreiben? (Die Lösung finden Sie auf der nächsten Folie)

a)
Geben Sie alle Kundendaten aus und sortieren Sie nach der Kundennummer absteigend.

b)
Geben Sie Name und Ort aller Kunden aus und sortieren Sie nach dem Ort alphabetisch.

Kundennr	Name	Ort
07642	König	Göttingen
63211	Schmitt	Hannover
85861	Müller	Hamburg

WEITERE BEISPIELE FÜR ODER-BY-STATEMENTS

a) Geben Sie alle Kundendaten aus und sortieren Sie nach der Kundennummer absteigend.

```
SELECT Kundennr, Name, Ort
FROM Kunde
ORDER BY Kundennr DESC;
```

Alternativ: SELECT *
FROM Kunde
ORDER BY Kundennr DESC;

Ausgabe:
85861 | Müller | Hamburg
63211 | Schmitt | Hannover
07642 | König | Göttingen

b) Geben Sie Name und Ort aller Kunden aus und sortieren Sie nach dem Ort alphabetisch.

```
SELECT Name, Ort
FROM Kunde
ORDER BY Ort ASC;
```

Ausgabe:
Müller | Hamburg
Schmitt | Hannover
König | Göttingen

Kundennr	Name	Ort
07642	König	Göttingen
63211	Schmitt	Hannover
85861	Müller	Hamburg

TABELLENVERWALTUNG - INSERT

- Mithilfe von INSERT können neue Datensätze hinzugefügt werden

- Syntax:

INSERT INTO Relation (Spalte1, Spalte2, ...)
VALUES (Wert1, Wert2, ...);

- Beispiel

INSERT INTO Kunde (Kundennr, Name, Ort)
VALUES (47111, 'Meier', 'Berlin');

Kundennr	Name	Ort
07642	König	Göttingen
63211	Schmitt	Hannover
85861	Müller	Hamburg
47111	Meier	Berlin

TABELLENVERWALTUNG - UPDATE

- Die UPDATE-Anweisung wird verwendet, um bereits vorhandene Daten in einer Tabelle zu ändern.
- Mit UPDATE werden bestehende Werte in einer oder mehreren Spalten angepasst.

- Syntax:

```
UPDATE Relation
SET Spalte1 = Wert1, Spalte2 = Wert2, ...
WHERE Bedingung;
```

- Beispiel:

```
UPDATE Kunde
SET Ort = 'Potsdam'
WHERE Kundennr = 47111;
```

Kundennr	Name	Ort
07642	König	Göttingen
63211	Schmitt	Hannover
85861	Müller	Hamburg
47111	Meier	Potsdam

TABELLENVERWALTUNG - DELETE

- Mittels DELETE können Datensätze aus Tabellen entfernt werden
- Eine oder mehrere Zeilen werden dauerhaft gelöscht

- Syntax:

DELETE FROM Relation
WHERE Bedingung;

- Beispiel

DELETE FROM Kunde
WHERE Kundennr = 47111;

- Hinweis: DELETE ohne WHERE würde alle Daten löschen

Kundennr	Name	Ort
07642	König	Göttingen
63211	Schmitt	Hannover
85861	Müller	Hamburg
~~47111~~	~~Meier~~	~~Berlin~~

WEITERE BEISPIELE FÜR INSERT-, UPDATE- UND DELETE-STATEMENTS

- Wie denken Sie sähen Statements für diese SQL-Abfragen aus? (Die Lösung finden Sie auf der nächsten Folie)

a) Fügen Sie eine neue Kundin mit folgenden Daten in die Tabelle ein: Kundennr: 57121; Name: Meier, Ort: Essen

b) Ändern Sie den Ort des Kunden König von Berlin auf Potsdam.

c) Löschen Sie den Kunden mit der Kundennummer 63211 aus der Tabelle.

Kundennr	Name	Ort
07642	König	Göttingen
63211	Schmitt	Hannover
85861	Müller	Hamburg

WEITERE BEISPIELE FÜR INSERT-, UPDATE- UND DELETE-STATEMENTS

a) Fügen Sie eine neue Kundin mit folgenden Daten in die Tabelle ein: Kundennr: 57121; Name: Meier, Ort: Essen

Lösung:
```
INSERT INTO Kunde (Kundennr, Name, Ort)
VALUES (57121, 'Meier', 'Essen');
```

b) Ändern Sie den Ort des Kunden König von Göttingen auf Potsdam.

Lösung:
```
UPDATE Kunde
SET Ort = 'Potsdam'
WHERE Name = 'König';
```

c) Löschen Sie den Kunden mit der Kundennummer 63211 aus der Tabelle.

Lösung:
```
DELETE FROM Kunde
WHERE Kundennr = 63211;
```

AGENDA

- Motivation
- Einführung in SQL
- **Weitere Informationen**

WEITERE HILFREICHE INFORMATIONSQUELLEN FÜR SQL

- W3School: https://www.w3schools.com/sql/default.asp
- https://studyflix.de/informatik/structured-query-language-606

VIEL SPAß BEI DEM ASSIGNMENT!

